

Bucket Sort e Radix Sort MSD

Bucket Sort

O **Bucket Sort** distribui os elementos em vários "baldes" (*buckets*) de acordo com seus valores. Depois, cada balde é ordenado individualmente e os seus elementos são reunidos no vetor original.

Funcionamento

- 1. Criar uma quantidade de baldes, normalmente relacionada ao tamanho do vetor.
- 2. Distribuir cada elemento no balde correspondente.
- 3. Ordenar cada balde separadamente.
- 4. Concatenar os baldes em ordem.

Nesta implementação, os valores devem ser `float` no intervalo `[0, 1)`. O índice do balde é calculado por `valor * n`.

Complexidade do Bucket Sort

Quando os elementos estão bem distribuídos, o custo médio é $O(n)$. No pior caso, todos os elementos podem cair no mesmo balde, resultando em $O(n^2)$.

Característica	Bucket Sort
Método	Distribuição
Melhor	$O(n)$
Médio	$O(n)$
Pior	$O(n^2)$
Memória	$O(n)$
Estável?	Depende da ordenação dos baldes
In-place?	Não

Implementação em C++ — sem STL

```
void bucketSort(float v[], int n) {
    if (n <= 0)
        return;

    float** baldes = new float*[n];
    int* tamanhos = new int[n];
    int capacidade = n;

    for (int i = 0; i < n; i++) {
        baldes[i] = new float[capacidade];
        tamanhos[i] = 0;
    }
}
```

```
}

for (int i = 0; i < n; i++) {
    int indice = (int)(v[i] * n);
    if (indice == n)
        indice = n - 1;
    baldes[indice][tamanhos[indice]++] = v[i];
}

for (int i = 0; i < n; i++) {
    for (int j = 1; j < tamanhos[i]; j++) {
        float atual = baldes[i][j];
        int k = j - 1;

        while (k >= 0 && baldes[i][k] > atual) {
            baldes[i][k + 1] = baldes[i][k];
            k--;
        }

        baldes[i][k + 1] = atual;
    }
}

int posicao = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < tamanhos[i]; j++)
        v[posicao++] = baldes[i][j];
    delete[] baldes[i];
}

delete[] baldes;
delete[] tamanhos;
}
```

Radix Sort MSD

O **Radix Sort MSD** (*Most Significant Digit*) ordena inteiros começando pelo dígito mais significativo. Os elementos são separados em grupos de acordo com esse dígito; depois, cada grupo é processado recursivamente usando o próximo dígito.

Funcionamento

1. Encontrar o maior valor para descobrir o maior divisor decimal.
2. Separar os elementos pelo dígito mais significativo.
3. Repetir a separação recursivamente para cada grupo, usando o próximo dígito.
4. Parar quando não houver mais dígitos para analisar.

Exemplo: para 329, o primeiro dígito analisado é o das centenas; depois, o das dezenas e, por fim, o das unidades. A implementação abaixo considera apenas inteiros não negativos.

Complexidade do Radix Sort MSD

Se n é a quantidade de elementos, d é a quantidade de dígitos e b é a base, a complexidade média é $O(d(n + b))$. Na base decimal, $b = 10$, então costuma ser representada por $O(dn)$. A memória auxiliar usada é $O(n)$.

Característica	Radix Sort MSD
Método	Distribuição por dígitos
Melhor	$O(d(n + b))$
Médio	$O(d(n + b))$
Pior	$O(d(n + b))$
Memória	$O(n + b)$
Estável?	Não necessariamente
In-place?	Não

Implementação em C++ — sem STL

```
void radixSortMSD(int v[], int n) {
    if (n <= 1)
        return;

    int maior = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] > maior)
            maior = v[i];
    }

    int divisor = 1;
    while (maior / divisor >= 10)
        divisor *= 10;

    int* auxiliar = new int[n];

    auto ordenar = [&](auto&& ordenar, int inicio, int fim, int div) -> void {
        if (inicio >= fim || div == 0)
            return;

        int contagem[10] = {0};

        for (int i = inicio; i <= fim; i++) {
            int digito = (v[i] / div) % 10;
            contagem[digito]++;
        }

        int inicioGrupo[10];
        inicioGrupo[0] = inicio;
        for (int digito = 1; digito < 10; digito++)
            inicioGrupo[digito] = inicioGrupo[digito - 1] + contagem[digito - 1];

        int proximaPosicao[10];
```

```

    for (int digito = 0; digito < 10; digito++)
        proximaPosicao[digito] = inicioGrupo[digito];

    for (int i = inicio; i <= fim; i++) {
        int digito = (v[i] / div) % 10;
        auxiliar[proximaPosicao[digito]++] = v[i];
    }

    for (int i = inicio; i <= fim; i++)
        v[i] = auxiliar[i];

    for (int digito = 0; digito < 10; digito++) {
        int grupoInicio = inicioGrupo[digito];
        int grupoFim = grupoInicio + contagem[digito] - 1;
        ordenar(ordenar, grupoInicio, grupoFim, div / 10);
    }
};

ordenar(ordenar, 0, n - 1, divisor);
delete[] auxiliar;
}

```

Colinha de complexidades

Considere n como a quantidade de elementos de entrada:

Complexidade	Significado	Exemplo comum
$O(1)$	Tempo constante: não depende do tamanho da entrada.	Acessar uma posição de um vetor
$O(\log n)$	Cresce lentamente; a cada passo, o problema é reduzido por uma divisão.	Busca binária
$O(\sqrt{n})$	Cresce mais que $O(\log n)$, mas mais lentamente que $O(n)$.	Testar divisores até a raiz quadrada
$O(n)$	Tempo linear: percorre a entrada uma vez.	Busca sequencial
$O(n \log n)$	Linear vezes logarítmica; comum em ordenações eficientes.	Merge Sort, Heap Sort
$O(d(n + b))$	Processa d dígitos, percorrendo n elementos e b grupos em cada etapa.	Radix Sort
$O(n + b)$	Memória proporcional aos elementos e aos grupos utilizados.	Memória auxiliar do Radix Sort
$O(n^2)$	Quadrática; geralmente envolve dois laços sobre a entrada.	Bubble Sort, Selection Sort
$O(n^3)$	Cúbica; geralmente envolve três laços sobre a entrada.	Algumas operações com matrizes

Complexidade	Significado	Exemplo comum
$O(2^n)$	Exponencial; dobra a cada novo elemento.	Algumas soluções recursivas de subconjuntos
$O(n!)$	Fatorial; cresce extremamente rápido.	Testar todas as permutações

Ordem de crescimento

Da melhor para a pior, em geral:

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Na análise de complexidade, constantes e termos de menor crescimento são ignorados. Por exemplo, $O(3n + 10)$ é simplificado para $O(n)$, e $O(n^2 + n)$ é simplificado para $O(n^2)$.